

AI PRACTICAL EXPERIMENTS - COMPREHENSIVE STUDY GUIDE

Table of Contents

1. Experiment 1: Graph Traversal (BFS & DFS)
 2. Experiment 2: A* Algorithm (8 Puzzle Problem)
 3. Experiment 3: Prim's Algorithm (MST)
 4. Experiment 4: N-Queens Problem
 5. Experiment 5: Simple Chatbot (Q&A System)
-

Experiment 1: Graph Traversal (BFS & DFS)

Title

Graph Traversal: Depth-First Search (DFS) and Breadth-First Search (BFS)

Aim

To understand and implement two fundamental graph traversal techniques: - DFS (Recursive approach) - explores deep into the graph - BFS (Iterative approach using queue) - explores level-by-level

Theory / Concept

What is Graph Traversal?

Graph traversal is the process of visiting each vertex (node) in a graph exactly once, following edges (connections) between vertices. It's the foundation for solving many problems like finding paths, detecting cycles, and exploring networks.

Key Concepts:

1. Depth-First Search (DFS)

- **Principle:** Uses a **stack** (LIFO - Last In First Out) data structure, implicitly via recursion
- **Approach:** Goes as deep as possible along each branch before backtracking
- **Order:** Visits a vertex, then recursively visits all its unvisited neighbors before moving to next vertex

- **Visual Intuition:** Like exploring a maze - follow one path completely, then backtrack to explore other paths

Working Principle:

1. Mark current vertex as visited
2. Print/Process the vertex
3. For each unvisited neighbor:
 - Recursively call DFS on that neighbor
4. Backtrack when no unvisited neighbors

2. Breadth-First Search (BFS)

- **Principle:** Uses a **queue** (FIFO - First In First Out) data structure
- **Approach:** Explores all neighbors of current vertex before moving to next level
- **Order:** Level-by-level exploration - visits all vertices at distance 1, then distance 2, etc.
- **Visual Intuition:** Like ripples in water - explore all nearby nodes before going far

Working Principle:

1. Add starting vertex to queue
2. Mark it as visited
3. While queue is not empty:
 - Remove front vertex from queue
 - Process/Print the vertex
 - Add all unvisited neighbors to queue and mark visited

Graph Representation: Adjacency List

Dictionary format:

```
graph = {
    0: [1, 2],      # Node 0 connects to nodes 1 and 2
    1: [0, 3],      # Node 1 connects to nodes 0 and 3
    2: [0, 3],      # Node 2 connects to nodes 0 and 3
    3: [1, 2]       # Node 3 connects to nodes 1 and 2
}
```

Time & Space Complexity:

Aspect	DFS	BFS
Time	$O(V + E)$	$O(V + E)$
Space	$O(V)$ - recursion stack	$O(V)$ - queue

Aspect	DFS	BFS
Best For	Deep exploration, topological sort	Shortest path, level-order

Algorithm

DFS Algorithm (Recursive)

Algorithm DFS(vertex, graph, visited):

Input: vertex (starting node), graph (adjacency list), visited (set)

Output: All vertices visited in DFS order

1. Add vertex to visited set
2. Print/Process vertex
3. For each neighbor in graph[vertex]:
 - If neighbor not in visited:
 - DFS(neighbor, graph, visited)
4. Return

BFS Algorithm (Iterative)

Algorithm BFS(start, graph):

Input: start (starting node), graph (adjacency list)

Output: All vertices visited in BFS order

1. Create empty set visited
 2. Create queue and add start node
 3. Mark start as visited
 4. While queue is not empty:
 - a. Remove front node from queue
 - b. Print/Process the node
 - c. For each neighbor of removed node:
 - If neighbor not visited:
 - Mark neighbor as visited
 - Add neighbor to queue
 5. Return
-

Code Explanation

Code Structure Breakdown:

Part 1: DFS Function

```
def dfs(vertex, graph, visited):
    visited.add(vertex)          # Mark current vertex as visited
    print(vertex, " ")          # Process the vertex

    for neighbour in graph[vertex]: # Check all neighbors
        if neighbour not in visited: # If not visited
            dfs(neighbour, graph, visited) # Recursive call
```

How it works: - Uses **recursion** (implicit stack) - When we call `dfs()` recursively, the function call is stored in the call stack - Each recursive call dives deeper into the graph - When no unvisited neighbors exist, function returns (backtrack) - Previous recursive calls continue exploring other branches

Part 2: BFS Function

```
def bfs(start, graph):
    visited = set()
    queue = deque([start])          # Initialize queue with start node
    visited.add(start)              # Mark start as visited

    while queue:                    # While queue has nodes
        vertex = queue.popleft()     # Remove and process front node
        print(vertex, " ")

        for neighbour in graph[vertex]: # Check all neighbors
            if neighbour not in visited: # If not visited
                visited.add(neighbour)   # Mark as visited
                queue.append(neighbour)  # Add to queue
```

How it works: - Uses **deque** (double-ended queue) from collections - `popleft()`: Remove from front (FIFO) - `append()`: Add to back - Queue ensures we process nodes level-by-level - All nodes at depth n are processed before depth $n+1$

Part 3: Graph Definition

```
graph = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}
```

This represents:

```

    0
   / \
  1   2
```

\ /
3

Part 4: Execution

```
print("DFS Traversal:")
dfs(0, graph, set())
# Output: 0 1 3 2 (may vary based on neighbor order)

print("\nBFS Traversal:")
bfs(0, graph)
# Output: 0 1 2 3 (level-by-level order)
```

Key Points / Important Notes

DFS Key Points:

- Uses **recursion** (implicit stack)
- Explores **deeply** before exploring breadth
- Order: **0** → **1** → **3** → **2** (depends on adjacency list order)
- Good for: topological sort, cycle detection, path finding
- Space: $O(V)$ for recursion stack (can overflow on very deep graphs)
- Memory: Efficient for sparse graphs

BFS Key Points:

- Uses **explicit queue** (deque)
- Explores **level-by-level**
- Order: **0** → **1** → **2** → **3** (guaranteed level order)
- Good for: shortest path, level-order traversal, distance calculation
- Space: $O(V)$ for queue
- Memory: Requires queue - may use more memory for dense graphs

Comparison Summary:

Feature	DFS	BFS
Data Structure	Stack (recursive)	Queue (explicit)
Memory Usage	Less in breadth, more in depth	More in breadth
Shortest Path	No guarantee	Yes (unweighted)
Cycle Detection	Yes	Yes
Easier to Implement	Yes (recursive)	Slightly complex
Use Case	Topological sort, DFS tree	Shortest path, MST

Viva Questions & Answers

Conceptual Questions:

Q1: What is the fundamental difference between DFS and BFS? A: DFS explores deep into a graph before backtracking (uses stack), while BFS explores level-by-level (uses queue). DFS = depth-first, BFS = breadth-first.

Q2: Why does DFS use recursion? A: Recursion automatically provides a stack mechanism. Each function call is pushed to the call stack, enabling the Last-In-First-Out (LIFO) behavior needed for DFS.

Q3: Why does BFS guarantee the shortest path in unweighted graphs? A: BFS explores all nodes at distance k before distance $k+1$, so the first path found is the shortest. DFS might find a longer path first.

Q4: What is the time complexity of both DFS and BFS? A: Both are $O(V + E)$, where V = vertices and E = edges, because each vertex and edge is visited exactly once.

Q5: What is the space complexity of DFS vs BFS? A: DFS = $O(V)$ for recursion stack. BFS = $O(V)$ for queue. But practical memory differs: DFS uses stack depth, BFS uses queue size.

Q6: Can DFS and BFS both detect cycles in a graph? A: Yes, both can detect cycles. If we visit a vertex already in the current path, there's a cycle.

Q7: What is the difference between visited set and recursion stack? A: Visited set tracks which nodes we've ever seen (global). Recursion stack tracks nodes in current path (local).

Q8: What happens if we don't use a visited set? A: Infinite loops! In graphs with cycles, we'd revisit nodes repeatedly, causing the program to run forever.

Q9: Why is DFS not suitable for finding shortest paths in unweighted graphs? A: DFS might go deep into one branch and find a long path before exploring shorter paths in other branches.

Q10: When would you prefer DFS over BFS? A: For topological sorting, detecting strongly connected components, and when memory is limited (stack < queue).

Code-Related Questions:

Q11: What does `visited.add(vertex)` do? A: It adds the vertex to a set of visited vertices to mark that we've already processed it.

Q12: What is deque and why use it instead of a list? A: Deque (double-ended queue) allows $O(1)$ operations at both ends. `popleft()` and `append()` are fast. Regular lists have $O(n)$ `pop(0)`.

Q13: Why call `visited.add(neighbour)` immediately in BFS before adding to queue? A: To prevent adding the same node to the queue multiple times, which would be inefficient.

Q14: What would happen if we process a vertex before marking it visited? A: If an unvisited neighbor is added to queue before marking visited, the same node might be added multiple times.

Q15: Explain for `neighbour in graph[vertex]`: A: This iterates through the adjacency list of the current vertex, getting all connected neighbors.

Difference-Based Questions:

Q16: DFS is recursive, BFS is iterative - why the difference? A: Recursion naturally provides stack behavior for DFS. BFS needs explicit queue, so iterative approach is more suitable.

Q17: Which one prints nodes in level-order? A: BFS prints in level-order (nodes at same distance from start). DFS prints in arbitrary order based on recursion.

Q18: Which traversal is better for maze solving? A: Both work, but DFS is simpler (recursive) and BFS guarantees shortest path.

Q19: In weighted graphs, which finds shortest path? A: Neither directly - need Dijkstra or Bellman-Ford for weighted graphs.

Q20: What's the space complexity difference in practice? A: DFS: stack depth = longest path. BFS: queue size = max width at any level. Wide graphs favor DFS.

Trick Questions:

Q21: If a graph is already a tree, do DFS and BFS behave the same? A: Time/space complexity same, but order of traversal differs (depth vs breadth).

Q22: Can you use BFS for disconnected graphs? A: Yes, but only reaches vertices reachable from start. Need to call BFS on each unvisited vertex separately.

Q23: What if you use a set instead of deque in BFS? A: It won't work correctly because sets are unordered. You need FIFO ordering, which only queue provides.

Q24: What's the difference between "visited" and "processed"? A: Visited = seen and added to queue. Processed = removed from queue and examined. Usually same, but important distinction in some problems.

Q25: Can DFS find all connected components? A: Yes, by calling DFS on each unvisited vertex in a loop, you find all components.

Common Mistakes

1. **Forgetting the visited set** - Results in infinite loops on cyclic graphs
 2. **Using pop(0) on list in BFS** - $O(n)$ operation; always use deque with popleft()
 3. **Not marking visited immediately** - Can add same node to queue multiple times
 4. **Assuming DFS gives shortest path** - It doesn't; use BFS for unweighted graphs
 5. **Confusion about when to mark visited** - Mark IMMEDIATELY when discovering, not when processing
 6. **Using list instead of set for visited** - $O(n)$ lookup; sets have $O(1)$ lookup
 7. **Not handling disconnected graphs** - Must iterate over all vertices if graph is not connected
 8. **Printing before marking visited in DFS** - Can print same node multiple times
 9. **Wrong recursion base case** - Should check visited in loop, not as function parameter
 10. **Misunderstanding adjacency list representation** - Is it undirected (both sides) or directed (one way)?
-

Quick Revision Summary

DFS: Recursive, stack-based, goes deep, $O(V+E)$, good for topological sort.
BFS: Queue-based, level-by-level, guarantees shortest path in unweighted graphs, $O(V+E)$. Both visit each vertex/edge once. DFS uses implicit recursion stack; BFS uses explicit deque. Neither is "better" - choose based on problem requirements: DFS for deep exploration, BFS for shortest path.

Experiment 2: A* Algorithm (8 Puzzle Problem)

Title

A* Search Algorithm: Solving the 8 Puzzle Problem

Aim

To understand and implement the A* algorithm, an informed search technique that combines actual cost and estimated cost to find optimal paths efficiently in problem-solving (using 8-puzzle as example).

Theory / Concept

What is A* Algorithm?

A* is a best-first search algorithm that finds the optimal path from start to goal by evaluating nodes based on a combination of: - **g(n)**: Actual cost from start to current node - **h(n)**: Estimated cost (heuristic) from current node to goal - **f(n) = g(n) + h(n)**: Total estimated cost

It selects the node with lowest f(n) to explore next.

Key Concepts:

1. What is the 8-Puzzle Problem?

- A 3×3 grid with tiles numbered 1-8 and one empty space (0)
- Goal: Rearrange tiles from any starting configuration to goal state
- Allowed moves: Slide adjacent tile into empty space (up, down, left, right)

Start State:

```
1 2 3
5 6 0  (0 = blank)
7 8 4
```

Goal State:

```
1 2 3
4 5 6
7 8 0
```

2. Heuristic Function h(n) The heuristic estimates distance to goal. Two common approaches:

a) Misplaced Tiles (Used in this code):

$h(n)$ = number of tiles not in goal position (excluding blank)

For above example: Count tiles in wrong position. - Fast to compute: $O(N^2)$ - Admissible: Never overestimates - Less informative: May explore more nodes

b) Manhattan Distance (Better but slower):

$h(n)$ = sum of $|current_row - goal_row| + |current_col - goal_col|$

- More accurate but slower to compute
- Also admissible

- Explores fewer nodes

3. Why A* is Better than Other Searches:

Algorithm	Use	Optimal	Informed
BFS	Unweighted graphs	Yes	No (explores all)
Dijkstra	Weighted graphs	Yes	No (explores many)
Greedy Best-First	Fast but risky	No (may miss optimal)	Yes
A*	General, weighted	Yes (if h is admissible)	Yes

4. How A* Works:

1. Maintain **priority queue** of nodes sorted by $f(n)$
2. Expand node with lowest $f(n)$
3. For each neighbor: Calculate $f(n) = g(n) + h(n)$
4. Add to queue if not visited
5. Continue until goal found or queue empty

5. Key Properties:

- **Admissibility:** If $h(n)$ = actual cost, A* finds optimal solution
- **Completeness:** A* will find solution if one exists
- **Optimality:** If $h(n)$ is admissible, solution is guaranteed optimal

6. Time & Space Complexity:

- **Time:** $O(b^d)$ where b = branching factor, d = depth (better than uninformed search)
- **Space:** $O(b^d)$ - must store all nodes in queue and visited set
- Exact complexity depends on heuristic quality

Algorithm

A* Algorithm for 8-Puzzle

```

Algorithm A_Star(start_state, goal_state):
    Input: start_state (initial puzzle configuration),
           goal_state (target configuration)
    Output: Goal state if found, "Not Found" otherwise

```

1. Initialize:
 - priority_queue (min-heap): sorted by $f(n)$
 - visited_set: empty set
 - directions: [(down), (left), (up), (right)]

2. Calculate $h = \text{heuristic}(\text{start_state})$
 - Add $(h, 0, \text{start_state}, \text{blank_x}, \text{blank_y})$ to `priority_queue`
 - Here $f=0+h, g=0$
3. While `priority_queue` is not empty:
 - a. Pop node with minimum $f(n)$ from queue
 (pop: $f_value, cost, \text{matrix}, x, y$)
 - b. Convert matrix to tuple and check if visited
 - If visited, continue to next iteration
 - Else, mark as visited
 - c. Check if goal_state reached
 - If yes, print "Goal Found" with cost, return
 - d. For each direction ($i = 0$ to 3):
 - Calculate new position: $\text{newX} = x + \text{direction}[i]$
 - Calculate new position: $\text{newY} = y + \text{direction}[i]$
 - If new position is valid (within bounds):
 - * Create new_state by copying current state
 - * Swap blank (x,y) with tile $(\text{newX}, \text{newY})$
 - * Convert new_state to tuple
 - * If new_state not visited:
 - ~ $g = cost + 1$ (increment cost)
 - ~ $h = \text{heuristic}(\text{new_state})$
 - ~ $f = g + h$
 - ~ Add $(f, g, \text{new_state}, \text{newX}, \text{newY})$ to queue
 4. If queue becomes empty without finding goal:
 - Print "Solution Not Found"
 - Return

Code Explanation

Part 1: Heuristic Function

```
def calculate_heuristic(mat):
    count = 0
    for i in range(N):           # N = 3 (3x3 grid)
        for j in range(N):
            if mat[i][j] != 0 and mat[i][j] != goal[i][j]:
                count += 1        # Count misplaced tiles
```

```
    return count
```

Logic: - Iterate through all positions - Skip blank (0) - Count tiles that are not in goal position - Returns number of misplaced tiles (0 when solved)

Part 2: Goal Check

```
def is_goal(mat):
    return mat == goal
```

Logic: Simple comparison of current state with goal state.

Part 3: Main A* Solver

```
def solve_astar(start, x, y):
    visited = set()
    pq = [] # priority queue (min-heap)

    h = calculate_heuristic(start)
    heapq.heappush(pq, (h, 0, start, x, y))
    # Format: (f=g+h, g, state, blank_x, blank_y)
```

Initialization: - visited: Tracks seen states (as tuples) - pq: Min-heap where smallest f(n) pops first - Initial h calculated for start state - g=0 (no moves yet), f=0+h

```
    # Moves: Down, Left, Up, Right
    row = [1, 0, -1, 0]
    col = [0, -1, 0, 1]
```

Direction Arrays: - row = [1, 0, -1, 0]: Down=1, Left=0, Up=-1, Right=0 - col = [0, -1, 0, 1]: Down=0, Left=-1, Up=0, Right=1

```
    while pq:
        f, cost, mat, x, y = heapq.heappop(pq)

        mat_tuple = tuple(tuple(row) for row in mat)

        if mat_tuple in visited:
            continue

        visited.add(mat_tuple)
```

Main Loop: - Pop node with minimum f(n) - Convert matrix to tuple (hashable for set) - Skip if already visited - Mark as visited

```
        if is_goal(mat):
            print("Goal State Reached!", cost)
            return
```

Goal Check: If we reached goal, return with cost.

```

for i in range(4): # Try all 4 moves
    newX = x + row[i]
    newY = y + col[i]

    if 0 <= newX < N and 0 <= newY < N:

```

Move Generation: - Calculate new blank position - Check if valid (within 3×3 grid)

```

new_mat = [row[:] for row in mat]
new_mat[x][y], new_mat[newX][newY] = \
    new_mat[newX][newY], new_mat[x][y]

```

Create New State: - Deep copy matrix: [row[:] for row in mat] - Swap blank with adjacent tile

```

new_tuple = tuple(tuple(r) for r in new_mat)

if new_tuple not in visited:
    g = cost + 1
    h = calculate_heuristic(new_mat)
    heapq.heappush(pq, (g + h, g, new_mat, newX, newY))

```

Add to Queue: - Convert to tuple (for visited check) - Skip if already visited
 - Calculate new cost: $g = \text{old_cost} + 1$ - Calculate heuristic: h - Calculate $f = g + h$ - Push to priority queue: $(f, g, \text{new_state}, x, y)$

Part 4: Execution

```

start = [[1, 2, 3],
         [5, 6, 0],
         [7, 8, 4]]

```

```

x, y = 1, 2 # blank position (row=1, col=2)

```

```

solve_astar(start, x, y)

```

The algorithm explores states in order of $f(n)$, finding shortest sequence of moves.

Key Points / Important Notes

A* Algorithm Points:

- **Optimal** if heuristic is admissible (never overestimates)
- **More efficient** than uninformed search (BFS, DFS)
- Uses **priority queue** (min-heap) for efficiency
- Balances **exploration** (g) and **estimation** (h)
- $f(n) = g(n) + h(n)$: lower f = better candidate

8-Puzzle Specifics:

- 3×3 grid = 362,880 possible states (9! permutations)
- Blank can move in 4 directions (up, down, left, right)
- Heuristic quality crucial: affects nodes explored
- Misplaced tiles: simple but less informative
- Manhattan distance: better heuristic (more accurate)

Implementation Points:

- Convert matrix to **tuple for hashing** (for visited set)
- Track both $f(n)$ and $g(n)$ in heap
- Check **bounds** before generating moves
- **Mark visited immediately** to avoid duplicate work
- Use **deep copy** (`[row[:] for row in mat]`) for new states

Complexity:

- Time: $O(b^d)$ with good heuristic ($b = 3$ avg for 8-puzzle)
- Space: $O(b^d)$ for storing states in visited set and queue
- Better heuristic = fewer nodes explored

Viva Questions & Answers

Conceptual Questions:

Q1: What does A* algorithm stand for? A: A-Star is an informed search algorithm. The name comes from the function $f(n) = g(n) + h(n)$, where A* evaluates nodes based on this function.

Q2: Explain the difference between $g(n)$, $h(n)$, and $f(n)$. A: $g(n)$ = actual cost from start to current node. $h(n)$ = estimated cost from current to goal (heuristic). $f(n)$ = total estimated cost = $g(n) + h(n)$.

Q3: Why is A* better than BFS for puzzle solving? A: BFS explores all nodes at each level equally. A* uses heuristic to guide search toward goal, exploring fewer nodes and finding solution faster.

Q4: What is an admissible heuristic? A: A heuristic that never overestimates the actual cost to reach goal. With admissible h , A* guarantees optimal solution.

Q5: Is misplaced tiles heuristic admissible? A: Yes, because each tile needs at least 1 move to correct, so `misplaced_count` = actual moves needed.

Q6: Which heuristic is better: misplaced tiles or Manhattan distance? A: Manhattan distance because it considers distance each tile needs to move (more informative). Misplaced tiles just counts quantity.

Q7: Can A* find solution if heuristic overestimates? A: A* will still find A solution, but not guaranteed optimal. Overestimating heuristics are called inadmissible.

Q8: Why convert matrix to tuple for visited set? A: Lists are not hashable (can't be used as set elements). Tuples are immutable and hashable, so they work in sets/dicts.

Q9: What is the goal state in 8-puzzle? A: Numbers 1-8 in order, blank (0) in bottom-right: `[[1,2,3], [4,5,6], [7,8,0]]`

Q10: How many possible states in 8-puzzle? A: $9! = 362,880$ states (all permutations of 9 positions). But only half are solvable from any starting position.

Code-Related Questions:

Q11: What does `heapq.heappush(pq, (f, g, new_mat, newX, newY))` do? A: Adds a node to priority queue. Heap maintains min-heap property: node with smallest $f(n)$ is always accessible via `pop`.

Q12: Why store `g` separately when `f = g + h` is stored? A: Because we need `g` to calculate `g+1` for child nodes. `f` alone doesn't tell us the actual cost.

Q13: What happens if we don't check visited before adding to queue? A: Same state added multiple times. Inefficient: process same state repeatedly, exponential increase in memory/time.

Q14: Explain deep copy `[row[:] for row in mat]`. A: Creates new list with copies of each row. `row[:]` copies each row. Without this, all states would reference same matrix.

Q15: What does `mat_tuple = tuple(tuple(row) for row in mat)` do? A: Converts 2D list to tuple of tuples: hashable representation for use in sets.

Difference-Based Questions:

Q16: Difference between Dijkstra and A* algorithm? A: Dijkstra uses only $g(n)$ (actual cost). A* uses $g(n) + h(n)$ (adds heuristic). A* is faster with good heuristic.

Q17: A* vs Greedy Best-First Search? A: Greedy only uses $h(n)$, not optimal. A* uses both $g(n)$ and $h(n)$, guarantees optimality with admissible h .

Q18: Why BFS not good for 8-puzzle? A: BFS explores all nodes at each depth level. For 8-puzzle with millions of states, BFS is extremely slow. A* is much faster.

Q19: What if heuristic is 0 for all states? A: $h(n) = 0$ means $f(n) = g(n)$ only. A* becomes Dijkstra's algorithm (expands in order of g , finds optimal but slower).

Q20: What if heuristic equals actual remaining distance? A: Perfect heuristic! A* explores only nodes on optimal path. Fewest nodes possible explored.

Trick Questions:

Q21: Is Manhattan distance heuristic admissible for 8-puzzle? A: Yes, because actual cost = Manhattan distance (each move covers 1 distance, you can't do better).

Q22: Can A* solve unsolvable puzzle configurations? A: No, it will exhaust all reachable states and return "Not Found". Only 50% of 8-puzzle configs are solvable from any start.

Q23: What's the f-value of the goal state? A: $f(\text{goal}) = g(\text{goal}) + h(\text{goal}) = \text{number_of_moves} + 0 = \text{optimal cost}$.

Q24: If two nodes have same $f(n)$, which A* picks? A: Depends on heap implementation. Usually the one added first, but not guaranteed (not a strict ordering).

Q25: Can you use A* for problems without clear heuristic? A: Yes, but with $h(n)=0$ it becomes Dijkstra (slow). Better to find domain-specific heuristic for efficiency.

Common Mistakes

1. **Using list instead of deque/heap** - Inefficient. Always use heapq for priority queue.
2. **Forgetting to convert matrix to tuple** - Lists aren't hashable; can't store in visited set.
3. **Not doing deep copy of matrix** - All states reference same object; mutations affect previous states.
4. **Overcomplicating heuristic** - Simple heuristics work fine; excessive computation defeats purpose.
5. **Checking visited AFTER popping, not before adding** - Less efficient; adds duplicates to queue.
6. **Using inadmissible heuristic** - May not find optimal solution. Always verify $h(n) \leq \text{actual_cost}$.
7. **Not tracking blank position correctly** - Blank position needed for move generation; easy to lose track.
8. **Calculating heuristic including blank** - Blank has no "goal" position; exclude from misplaced count.
9. **Wrong state representation** - Must be consistent and hashable. Lists won't work in sets.
10. **Not handling invalid moves** - Must check bounds ($0 \leq \text{newX}, \text{newY} < N$) before creating new state.

11. **Printing wrong information** - Should print cost when goal reached, not g variable.
 12. **Using cost before incrementing** - $g(\text{child}) = g(\text{parent}) + 1$, not $g(\text{parent})$.
-

Quick Revision Summary

A* Algorithm: Combines actual cost $g(n)$ and estimated cost $h(n)$ to guide search efficiently. $f(n) = g(n) + h(n)$. Uses priority queue (min-heap) to always expand most promising node. For 8-puzzle: heuristic = misplaced tiles (or Manhattan distance). Convert matrix to tuple for visited set. A* guarantees optimal solution if h is admissible (never overestimates). Much faster than BFS for large search spaces.

Experiment 3: Prim's Algorithm (Minimum Spanning Tree)

Title

Prim's Algorithm: Finding Minimum Spanning Tree (MST)

Aim

To understand and implement Prim's algorithm for finding a Minimum Spanning Tree in a weighted undirected graph, which connects all vertices with minimum total edge weight.

Theory / Concept

What is a Spanning Tree?

- A spanning tree of a graph is a subgraph that:
 - Includes all vertices
 - Is a tree (no cycles)
 - Has exactly $V-1$ edges (for V vertices)
 - Is connected

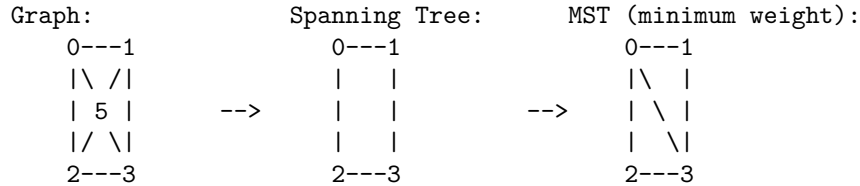
Example: Graph with 4 vertices has $2^{(V-1)}$ possible spanning trees.

What is Minimum Spanning Tree (MST)?

- A spanning tree where the sum of edge weights is minimum

- Used in: network design, road connections, circuit design
- May not be unique (multiple MSTs with same weight possible)

Visual Example:



Key Concepts:

1. How Prim's Algorithm Works: Strategy: Greedy approach - always pick the minimum weight edge connecting tree to new vertex.

Process: 1. Start with any vertex (usually 0) 2. Maintain a set of "selected vertices" (initially just start) 3. Repeatedly find minimum weight edge connecting: - A selected vertex (in tree) - To an unselected vertex (outside tree) 4. Add that edge and new vertex to tree 5. Continue until all V-1 edges added

2. Why It's Greedy: Each step picks locally optimal choice (minimum edge). Greedy choice works because of MST property (matroid structure).

3. Visual Walkthrough:

Graph (weights on edges):

```

0 ↔ 1 (weight 19)
0 ↔ 2 (weight 5)   ← lightest from 0
1 ↔ 2 (weight 5)
1 ↔ 3 (weight 9)
2 ↔ 3 (weight 1)   ← lightest overall
2 ↔ 4 (weight 6)
3 ↔ 4 (weight 1)   ← lightest overall

```

Step 1: Select vertex 0. Tree = {0}
 Find minimum edge from 0 to others
 → 0-2 (weight 5) ← smallest
 Add 2 to tree

Step 2: Tree = {0, 2}. Find minimum edge from {0,2} to others
 Edges: 0-1(19), 2-1(5), 2-3(1), 2-4(6)
 → 2-3 (weight 1) ← smallest
 Add 3 to tree

Step 3: Tree = {0, 2, 3}. Find minimum edge from tree to others
 Edges: 0-1(19), 2-1(5), 2-4(6), 3-1(9), 3-4(1)

→ 3-4 (weight 1) ← smallest
Add 4 to tree

Step 4: Tree = {0, 2, 3, 4}. Find minimum edge from tree to others
Edges: 0-1(19), 2-1(5), 3-1(9)
→ 2-1 (weight 5) ← smallest
Add 1 to tree

Result: MST edges: {0-2:5, 2-3:1, 3-4:1, 2-1:5}
Total weight: 5+1+1+5 = 12

4. Algorithm Characteristics:

Aspect	Details
Type	Greedy algorithm
Structure	Builds tree incrementally
Optimal	Yes, guarantees MST
Greedy Choice	Always pick minimum edge to new vertex
Works on	Connected, weighted, undirected graphs

5. Time & Space Complexity:

Implementation	Time	Space
Naive (code used)	$O(V^2)$	$O(V^2)$ for adjacency matrix
With Binary Heap	$O((V+E) \log V)$	$O(V+E)$ for adjacency list
With Fibonacci Heap	$O(E + V \log V)$	$O(V+E)$

The code uses $O(V^2)$ implementation: for each edge added ($V-1$), scans all vertices (V) to find minimum.

Algorithm

Prim's Algorithm - Pseudocode

Algorithm Prim(graph):

Input: Connected weighted undirected graph

Output: Minimum Spanning Tree edges and total weight

1. Initialize:

- selected_node = [False] * V // All vertices unselected initially
- selected_node[0] = True // Start with vertex 0
- edges_added = 0

```

- total_weight = 0

2. While edges_added < V - 1:    // Need V-1 edges for V vertices
  a. minimum_weight = INFINITY
    starting_vertex = -1
    ending_vertex = -1

  b. For each selected vertex i:
    For each unselected vertex j:
      If there's edge (i,j) with weight W:
        If W < minimum_weight:
          minimum_weight = W
          starting_vertex = i
          ending_vertex = j

  c. Print "Edge: {starting_vertex} -> {ending_vertex} : {weight}"

  d. selected_node[ending_vertex] = True

  e. edges_added += 1

  f. total_weight += minimum_weight

3. Print total_weight

End Algorithm

```

Code Explanation

Part 1: Initialization

```

INF = 99999999 # Represent infinity
N = 5          # Number of vertices

G = [          # Adjacency matrix (5x5)
  [0, 19, 5, 0, 0],
  [19, 0, 5, 9, 2],
  [5, 5, 0, 1, 6],
  [0, 9, 1, 0, 1],
  [0, 2, 6, 1, 0]
]

```

Graph Representation: - Adjacency Matrix: $G[i][j]$ = weight of edge $i \rightarrow j$
 - $G[i][j] = 0$ means no edge (except diagonal) - Undirected: $G[i][j] = G[j][i]$

Example: $G[0][2] = 5$ means edge 0-2 has weight 5

```

selected_node = [False] * N    # Track which nodes in MST
selected_node[0] = True        # Start from vertex 0
no_edge = 0                    # Count edges added

```

Tracking: - `selected_node[i]` = True if vertex `i` is in current MST - Initially only vertex 0 is in tree - `no_edge` counts edges added (stops at `V-1`)

Part 2: Main Loop

```

print("Edge : Weight\n")

```

```

while (no_edge < N - 1):    # Need N-1 edges for N vertices

```

Loop runs exactly `V-1` times, adding one edge each iteration.

Part 3: Finding Minimum Edge

```

minimum = INF
a = 0
b = 0

for i in range(N):          # Check all selected vertices
    if(selected_node[i]):    # If vertex i is in tree
        for j in range(N):  # Check all other vertices
            if((not selected_node[j] and G[i][j] != 0)):
                # If j is NOT in tree AND edge i-j exists
                if minimum > G[i][j]: # If weight is smaller
                    minimum = G[i][j]
                    a = i           # Remember edge i-j
                    b = j

```

Logic: - Loop through all selected vertices `i` (in tree) - For each, check all unselected vertices `j` (outside tree) - Find edge with minimum weight connecting tree to outside - Store minimum edge as `a-b`

Part 4: Adding Edge and Vertex

```

print(f'{a} -> {b} : {G[a][b]}')    # Print selected edge
no_edge += 1                         # One more edge added
selected_node[b] = True               # Add vertex b to tree

```

Actions: - Print the edge and its weight - Increment edge counter - Mark endpoint vertex as selected

Part 5: Execution Flow

Initial:

```

selected_node = [T, F, F, F, F]    // Only 0 selected

```

Iteration 1: - Check vertex 0 to all others - Find minimum: 0-2 (weight 5) -
 Print: "0 -> 2 : 5" - Mark: `selected_node[2] = True`

Iteration 2:

`selected_node = [T, F, T, F, F]` // 0 and 2 selected

- Check vertices 0 and 2
- Find minimum: 2-3 (weight 1)
- Print: "2 -> 3 : 1"
- Mark: `selected_node[3] = True`

Continue until all $V-1$ edges added.

Key Points / Important Notes

Prim's Algorithm Points:

- **Greedy approach:** Always pick locally optimal (minimum edge)
- **Guarantees MST:** Works because of matroid property
- **Builds incrementally:** Expands tree by one edge each iteration
- **Simple to implement:** $O(V^2)$ naive version straightforward
- **Works with any starting vertex:** MST total weight same regardless
- **For connected graphs only:** If disconnected, finds MST of component containing start

Implementation Points:

- Use **adjacency matrix** for dense graphs
- Check **both directions** (though undirected symmetry handles it)
- $G[i][j] = 0$ means no edge (not zero weight!)
- Use large number (INF) to represent no edge
- Always check $G[i][j] \neq 0$ before considering edge

Complexity Analysis:

- **Time:** $O(V^2)$ - outer loop V iterations, inner loop V iterations each
- **Space:** $O(V^2)$ - store adjacency matrix
- **For dense graphs:** $O(V^2)$ optimal
- **For sparse graphs:** Can improve with heaps to $O(E \log V)$

Comparison with Kruskal's Algorithm (other MST approach):

Aspect	Prim's	Kruskal's
Approach	Vertex-based	Edge-based
Data Structure	Adjacency matrix	Sorted edges

Aspect	Prim's	Kruskal's
Time (naive)	$O(V^2)$	$O(E \log E)$
Greedy	Add vertex with min edge	Add min edge if no cycle
Better for	Dense graphs	Sparse graphs

Viva Questions & Answers

Conceptual Questions:

Q1: What is a spanning tree? A: A subgraph that includes all vertices, has no cycles, and is connected. It has exactly $V-1$ edges for V vertices.

Q2: What is a Minimum Spanning Tree (MST)? A: A spanning tree where the sum of all edge weights is minimum. Multiple MSTs possible with same total weight.

Q3: Why is Prim's algorithm greedy? A: Each step picks the locally optimal choice (minimum weight edge). Greedy works because MST has optimal substructure property.

Q4: How does Prim's algorithm guarantee optimality? A: Because of the cut property: for any cut of graph, minimum weight edge crossing cut is in some MST.

Q5: What starting vertex should you use in Prim's? A: Any vertex works. The MST total weight is same regardless of starting vertex (for connected graphs).

Q6: Can Prim work on directed graphs? A: Not in standard form. Prim's is for undirected graphs. For directed graphs, use other algorithms.

Q7: What if graph has multiple edges with same weight? A: Multiple MSTs may exist with same total weight. Prim finds one of them (which depends on edge selection order).

Q8: Difference between Prim and Kruskal? A: Prim builds from vertices (start with one vertex, expand). Kruskal builds from edges (sort edges, pick minimum if no cycle).

Q9: When is Prim's better than Kruskal's? A: For dense graphs (many edges), Prim's $O(V^2)$ better than Kruskal's $O(E \log E)$. For sparse graphs, Kruskal better.

Q10: What does MST weight represent? A: Minimum cost to connect all vertices. In network design: minimum cable needed. In roads: minimum total distance.

Code-Related Questions:

Q11: What does $G[i][j] = 0$ mean? A: In this code, no edge exists between vertices i and j . Note: This is context-specific; usually 0 weight edge is valid.

Q12: Why check $G[i][j] \neq 0$ before considering edge? A: To skip non-existent edges. In adjacency matrix representation, 0 indicates no edge.

Q13: What does `selected_node[i]` track? A: Boolean array: True if vertex i is already in MST, False if still outside.

Q14: Why initialize all to False except `selected_node[0] = True`? A: Prim's starts with one vertex (vertex 0). Other vertices to be added incrementally.

Q15: What does `minimum = INF` do? A: INF is very large number (99999999). Initialize minimum to INF so any real edge weight is smaller (minimum found correctly).

Difference-Based Questions:

Q16: Prim adds vertices, Kruskal adds edges - what's the implication?

A: Prim ensures connectivity at each step (tree property). Kruskal might have disconnected components until last edges added.

Q17: Why is adjacency matrix used instead of list? A: Because $O(V^2)$ scan for minimum edge is natural with matrix. With list, would need to store and sort edges separately.

Q18: Prim's time $O(V^2)$ vs Kruskal's $O(E \log E)$ - when each better?

A: Prim: dense graphs ($E \sim V^2$). Kruskal: sparse graphs ($E \sim V$).

Q19: If all edge weights equal, what happens? A: Any spanning tree is also MST. Prim finds one valid MST.

Q20: What if graph is disconnected? A: Prim finds MST of component containing start vertex. Other components not included.

Trick Questions:

Q21: Is MST unique? A: Not always. If multiple edges have same weight, multiple MSTs possible with same total weight.

Q22: Can you start Prim from any vertex on disconnected graph?

A: Yes, but only connects component containing start. Must run separately on other components.

Q23: What's the maximum weight edge in MST? A: Hard to say without seeing graph. But: all edges in MST are locally minimal (min crossing cut).

Q24: Can MST have a very heavy edge? A: Yes! If that edge is necessary to connect two components, it might be in MST despite high weight.

Q25: Does Prim's work if edge weights are negative? A: Yes, MST property still holds. However, "minimum" is still about sum, which could be very negative.

Common Mistakes

1. **Confusing 0 weight with no edge** - In this code, 0 means no edge (not zero weight). Dangerous assumption.
 2. **Starting from wrong vertex** - While any vertex works, must mark it selected initially.
 3. **Not checking `!selected_node[j]`** - Can add vertex twice, causing wrong edges.
 4. **Not checking `G[i][j] != 0`** - Considers non-existent edges, may pick wrong minimum.
 5. **Forgetting to mark vertex as selected** - `selected_node[b] = True` required or loop will repeat.
 6. **Wrong loop termination** - Need exactly $V-1$ edges, not V edges. Off-by-one error common.
 7. **Not initializing minimum to INF** - If initialized to 0, might pick edge with weight 0 incorrectly.
 8. **Assuming graph is always connected** - Prim works on component, not full graph if disconnected.
 9. **Using adjacency list incorrectly** - Adjacency matrix more straightforward for this algorithm.
 10. **Not printing total weight** - Should print sum of all edge weights at end.
 11. **Edge case: Graph with 1 vertex** - No edges needed ($V-1 = 0$). Code handles correctly.
 12. **Assuming unique MST** - Multiple MSTs possible. Prim finds one, not necessarily "the" MST.
-

Quick Revision Summary

Prim's Algorithm: Greedy approach to find MST. Start with one vertex, repeatedly add minimum weight edge connecting tree to new vertex until all V vertices included ($V-1$ edges). Time $O(V^2)$ naive, better with heaps. Builds tree incrementally. Guaranteed optimal due to cut property. Start from any vertex; MST weight same. Use for dense graphs. Different from Kruskal's which sorts edges first.

Experiment 4: N-Queens Problem (Backtracking & Branch and Bound)

Title

N-Queens Problem: Solving with Backtracking and Branch & Bound Techniques

Aim

To understand and implement two constraint satisfaction solving techniques:

1. **Backtracking (4 Queens):** Basic approach with recursive exploration
2. **Branch and Bound (8 Queens):** Optimized approach with constraint maintenance

Theory / Concept

What is the N-Queens Problem?

- Place N queens on N×N chessboard
- Constraint: No two queens attack each other
- Queens attack: same row, same column, same diagonal
- Goal: Find all valid placements (or any one solution)

Example: 4-Queens

Solution 1:	Solution 2:
. Q . .	. . Q .
. . . Q	Q . . .
Q . . .	. . . Q
. . Q .	. Q . .

Key Concepts:

1. Attack Vectors: Queens attack each other if: - **Same Row:** Queen[i][j] and Queen[i][k] (j ≠ k) - **Same Column:** Queen[i][j] and Queen[k][j] (i ≠ k) - **Diagonal 1:** Queen[i][j] and Queen[i+d][j+d] (both diagonals) - **Diagonal 2:** Queen[i][j] and Queen[i+d][j-d] (anti-diagonal)

2. Backtracking Approach: Principle: Explore solutions recursively. When constraint violated, backtrack (undo).

Strategy: - Place queens column by column - For each column, try each row - Check if position safe (no existing queen attacks) - If safe, place queen and move to next column - If unsafe or dead end, backtrack and try next row - Backtrack = remove queen, try different position

Complexity: - Time: O(N!) worst case (try all permutations) - Space: O(N) for recursion stack - Pruning helps: eliminates many invalid branches early

3. Branch and Bound Approach: Enhancement over Backtracking:

- Maintain constraint-checking structures (lookup arrays) - Faster safety check: $O(1)$ instead of $O(N)$ - Same logic, much faster execution

Optimization Technique: - Instead of checking all N previous queens for each candidate position - Maintain three boolean arrays: - `rowlookup[i]`: Is row i occupied? - `slashlookup[d]`: Is diagonal d occupied? - `backslashlookup[d]`: Is anti-diagonal d occupied?

Diagonal Encoding: For position (row, col): - Diagonal 1 index = row + col (stays same on main diagonal) - Diagonal 2 index = row - col + (N-1) (stays same on anti-diagonal)

4. Visual Walkthrough (4 Queens):

```
Place Q in col 0:
Try row 0: [Q . . .]
  Place Q in col 1:
    Try row 0: Attacked by col 0 Q - skip
    Try row 1: [Q . . .] with Q at (1,1)
      Check: different row , different diagonals
      Place Q: [Q Q . .]
    Place Q in col 2:
      Try row 0: Attacked - skip
      Try row 1: Attacked - skip
      Try row 2: [Q Q Q .] - Attacked - skip
      Try row 3: [Q Q . Q] - Check diagonals... backtrack
    No solution this path, backtrack
  Try row 2: [Q . Q .]
    Check: different row , different diagonals
    Place Q: [Q . Q .]
  Place Q in col 3:
    Try row 1: [Q . Q Q]
      Check: row 1 not used, diagonals safe? YES!
      SOLUTION FOUND: row placement = [0, 2, 3, 1]
      But algorithm prints board and continues finding others

(Process continues until all solutions explored)
```

Algorithm

Backtracking Algorithm (4-Queens)

```
Algorithm N_Queens_Backtracking(board, col, N):
  Input: board (N×N matrix, 0=empty, 1=queen)
         col (current column to place queen)
```

```

        N (board size)
    Output: True if solution found, False if not

1. Base Case:
    If col >= N:
        Solution found (all queens placed)
        Return True

2. Try each row in current column:
    For row = 0 to N-1:
        If is_safe(row, col, board):
            a. Place queen: board[row][col] = 1

                b. Recursively try next column:
                    If N_Queens(board, col+1) returns True:
                        Return True (solution found)

            c. Backtrack: board[row][col] = 0
                (undo placement, try next row)

3. If no row works:
    Return False (dead end, backtrack to previous column)

End Algorithm

Algorithm is_safe(row, col, board):
    Input: row, col (position to check)
           board (current state)
    Output: True if position safe, False if attacked

1. Check row to left:
    For i = 0 to col-1:
        If board[row][i] == 1:
            Return False (queen exists in row)

2. Check upper-left diagonal:
    i, j = row, col
    While i > 0 and j > 0:
        i -= 1, j -= 1
        If board[i][j] == 1:
            Return False

3. Check lower-left diagonal:
    i, j = row, col
    While i < N and j > 0:
        i += 1, j -= 1

```

```

        If board[i][j] == 1:
            Return False

4. If all checks pass:
    Return True (position is safe)

End Algorithm

Branch and Bound Algorithm (8-Queens)

Algorithm N_Queens_BranchAndBound(col, constraints):
    Input: col (current column)
           constraints (lookup arrays for rows/diagonals)
    Output: True if solution found, False if not

1. Base Case:
    If col >= N:
        Solution found
        Print board
        Return True

2. Try each row:
    For row = 0 to N-1:
        If is_safe(row, col, constraints):
            a. Place queen: board[row][col] = 1

            b. Update constraints:
                rowlookup[row] = True
                slashlookup[diagonal1_code] = True
                backslashlookup[diagonal2_code] = True

            c. Recursively solve next column:
                If N_Queens_Branch(col+1, constraints) returns True:
                    Return True

            d. Backtrack: Undo placements:
                board[row][col] = 0
                rowlookup[row] = False
                slashlookup[diagonal1_code] = False
                backslashlookup[diagonal2_code] = False

3. Return False (dead end)

End Algorithm

Algorithm is_safe(row, col, constraints):

```

```

    Input: row, col (position)
           constraints (lookup arrays)
    Output: True if safe, False if attacked

1. Check if any constraint violated:
    If rowlookup[row] OR slashlookup[code1] OR backslashlookup[code2]:
        Return False

2. If all constraints satisfied:
    Return True

End Algorithm

```

Code Explanation

Part 1: Backtracking Approach (4 Queens)

Safety Check Function

```

def is_safe(row, col, board):
    # Check row to the left
    for i in range(N):
        if(board[row][i] == 1): # If queen exists in this row
            return False

    # Check upper-left diagonal
    i, j = row, col
    while(i > 0 and j > 0):
        if(board[i][j] == 1): # If queen on diagonal
            return False
        i -= 1
        j -= 1

    # Check lower-left diagonal
    i, j = row, col
    while(i < N and j > 0):
        if(board[i][j] == 1): # If queen on anti-diagonal
            return False
        i += 1
        j -= 1

    return True

```

Logic: - Only check left side (queens to right haven't been placed yet) - Row check: scan from col 0 to col-1 for queens in same row - Diagonal 1: move up-left until out of bounds - Diagonal 2: move down-left until out of bounds

Recursive Solver

```
def solve(board, col):
    if col >= N:          # Base case: all columns filled
        return True      # Solution found

    for i in range(N):    # Try each row in current column
        if is_safe(i, col, board): # If position safe
            board[i][col] = 1    # Place queen
            if(solve(board, col+1)): # Try next column
                return True      # Solution found in recursion
            board[i][col] = 0    # Backtrack: remove queen

    return False          # No solution in this branch
```

Flow: 1. Base case: col reaches N $\rightarrow$ all queens placed $\rightarrow$ solution found 2. Recursive case: try each row 3. If safe, place queen and recurse on next column 4. If recursion returns True, solution exists, propagate True 5. If no row works (all recursions fail), backtrack and return False

Main Function

```
def main():
    board = [[0]*N for _ in range(N)] # Empty N*N board

    if solve(board, 0): # Start from column 0
        for row in board:
            print(row)   # Print solution
    else:
        print("No Solution")
```

main()

Part 2: Branch & Bound Approach (8 Queens)

Initialization

N = 8

board = [[0]*N for _ in range(N)]

slashcode = [[r+c for c in range(N)] for r in range(N)]

backslashcode = [[r-c+(N-1) for c in range(N)] for r in range(N)]

Diagonal Codes: - slashcode[r][c] = r + c: Main diagonal identifier -
backslashcode[r][c] = r - c + (N-1): Anti-diagonal identifier

Why this works: - All cells on same diagonal have same code - Example:
(0,0), (1,1), (2,2) all have r+c = 0,2,4... $\rightarrow$ same diagonal

Lookup Arrays

```
rowlookup = N*[False]           # N rows
slashlookup = (2*N-1)*[False]   # 2N-1 possible main diagonals
backslashlookup = (2*N-1)*[False] # 2N-1 possible anti-diagonals
```

Sizes: - Rows: N possible row indices - Main diagonals: can be 0 to 2N-2 (range of r+c) - Anti-diagonals: can be 0 to 2N-2 (range of r-c+(N-1))

Safety Check (Optimized)

```
def is_safe(row, col, slashcode, backslashcode,
            rowlookup, slashlookup, backslashlookup):
    if(rowlookup[row] or
        slashlookup[slashcode[row][col]] or
        backslashlookup[backslashcode[row][col]]):
        return False # Any constraint violated
    return True      # All constraints satisfied
```

Efficiency: O(1) lookup instead of O(N) scanning!

Recursive Solver

```
def solve_util(board, col, slashcode, backslashcode,
               rowlookup, slashlookup, backslashlookup):
    if(col >= N):
        return True # Solution found

    for i in range(N):
        if(is_safe(i, col, slashcode, backslashcode,
                  rowlookup, slashlookup, backslashlookup)):

            board[i][col] = 1
            rowlookup[i] = True
            slashlookup[slashcode[i][col]] = True
            backslashlookup[backslashcode[i][col]] = True

            if(solve_util(board, col+1, ...)):
                return True

    # Backtrack: undo all placements
    board[i][col] = 0
    rowlookup[i] = False
    slashlookup[slashcode[i][col]] = False
    backslashlookup[backslashcode[i][col]] = False

    return False
```


Key Improvement: Update constraint arrays with each placement. Safety check becomes constant time!

Key Points / Important Notes

Backtracking Fundamentals:

- **Recursive exploration** of solution space
- **Pruning:** Cut branches that violate constraints
- **Undo operations** when backtracking
- **Time:** $O(N!)$ worst case, much better with pruning
- **Space:** $O(N)$ for recursion stack
- For 4-Queens: reasonable ($4! = 24$ permutations)
- For N-Queens: only 92 solutions for $N=8$

Branch & Bound Optimization:

- **Constraint maintenance** via lookup arrays
- **$O(1)$ safety check** instead of $O(N)$ scanning
- Same recursion logic, much faster execution
- For 8-Queens: practical even though $O(N!)$ theoretical
- Scales better than pure backtracking
- Handles $N=8$ efficiently (millions of branches pruned)

Board Representation:

- 2D array: $\text{board}[\text{row}][\text{col}] = 1$ (queen) or 0 (empty)
- Place one queen per column (constraint reduces search space)
- Check left side only (right side not filled yet)
- Can also use 1D array to store column for each row

Key Differences:

Aspect	Backtracking	Branch & Bound
Safety Check	$O(N)$ scan	$O(1)$ lookup
Data Structure	Board only	Board + 3 lookup arrays
Time (practical)	Slower for large N	Much faster
Memory	$O(N)$ stack	$O(N)$ stack + arrays
Suitable for	Small N (4)	Medium N (8)
Complexity	$O(N!)$ theoretical	$O(N!)$ but pruned efficiently

Viva Questions & Answers

Conceptual Questions:

Q1: What is the N-Queens problem? A: Place N queens on N×N chess-board such that no two queens attack each other. Queens attack along rows, columns, and both diagonals.

Q2: Why is N-Queens hard? A: Search space is N! (all permutations). Only small fraction are valid. Must explore many branches before pruning.

Q3: What does backtracking mean? A: When you reach dead end, undo last decision and try alternative. Recursively explore different choices.

Q4: How does backtracking reduce search space? A: Constraint checking prunes invalid branches early. Don't continue exploring branches that violate constraints.

Q5: What is Branch and Bound? A: Optimization technique that adds bounds/constraints to prune branches faster. Maintains data structures for quick constraint checking.

Q6: Why place queens column-by-column? A: Reduces search space. Guarantees one queen per column automatically. Only need check N² positions instead of N! permutations.

Q7: Is backtracking solution guaranteed to find answer? A: Yes, if solution exists. Explores all valid combinations. (For N-Queens, solutions exist for all N ≥ 4.)

Q8: What's the time complexity of N-Queens? A: O(N!) worst case (try all permutations). But pruning makes it much faster (approximately O(N!) / factorial_pruning).

Q9: Can N-Queens be solved for any N? A: No. For N=2 and N=3, no solution. For N ≥ 4, solution exists. As N increases, number of solutions varies.

Q10: Why is 8-Queens famous? A: First posed in 1848. Interesting challenge for backtracking algorithms. Has 92 distinct solutions. Benchmark for AI algorithms.

Code-Related Questions:

Q11: What does `board[i][col] = 1` do? A: Places a queen at position (i, col) on the board.

Q12: Why check for `i in range(N)` instead of already-filled columns? A: Because we're filling column by column left to right. Only previous columns have queens (to the left).

Q13: What does the backtracking line `board[i][col] = 0` accomplish? A: Removes the queen (undoes placement) before trying next row. Allows reuse

of position for different solution attempt.

Q14: In Branch & Bound, why update lookup arrays? A: To quickly check constraints in $O(1)$ without scanning board. `rowlookup[i]` tells immediately if row `i` is occupied.

Q15: What are slashcode and backslashcode? A: Diagonal identifiers. All cells on same diagonal share same code. Allows $O(1)$ diagonal conflict detection.

Difference-Based Questions:

Q16: Difference between backtracking and brute force? A: Brute force tries all $N!$ arrangements. Backtracking prunes invalid branches early, explores much fewer combinations.

Q17: Backtracking vs Branch & Bound? A: Same recursion logic. Branch & Bound adds constraint lookup arrays for faster safety checks ($O(1)$ vs $O(N)$).

Q18: Why is Branch & Bound better for 8-Queens? A: $O(1)$ safety check vs $O(N)$ scanning. For 8-Queens, difference between milliseconds and seconds.

Q19: Can you use 1D array instead of 2D for board? A: Yes, only one queen per column anyway. `board[col] = row` represents position. More memory efficient.

Q20: Why only check left of current position? A: Right side not filled yet (no queens there). Only threats from already-placed queens (on left).

Trick Questions:

Q21: If you forget to backtrack, what happens? A: Algorithm finds only first solution (or none if backtrack truly forgotten). Doesn't explore other branches.

Q22: What if you place queens row-by-row instead of column-by-column? A: Still works, but same algorithm logic (just transposed). Column-by-column is conventional.

Q23: For $N=4$, how many solutions exist? A: 2 solutions (4 distinct placements that work).

Q24: For $N=8$, total possible arrangements? A: $8! = 40,320$. But only 92 are valid solutions.

Q25: Can you parallelize N-Queens solving? A: Yes, explore different columns in parallel. But backtracking inherently sequential (each level depends on previous).

Common Mistakes

1. **Forgetting backtrack step** - `board[i][col] = 0` essential. Without it, board state corrupted.
2. **Checking entire board instead of just left** - Unnecessary. Right side empty; only check left for efficiency.
3. **Not initializing board** - Must start with all zeros.
4. **Wrong diagonal check logic** - Easy to confuse bounds and directions for diagonals. Trace through carefully.
5. **Checking `board[i][j] == 1` when should check constraint array** - Branch & Bound uses lookup arrays, not board scanning.
6. **Forgetting to undo constraint updates** - If backtracking, must reset all lookup arrays too.
7. **Wrong diagonal code calculation** - `slashcode = r+c`, `backslashcode = r-c+(N-1)`. Off-by-one errors common.
8. **Using wrong bounds for loops** - Constraints have size $2N-1$, not N . Array index out of bounds.
9. **Not handling base case correctly** - `if col >= N` not `col == N` (though minor difference).
10. **Checking already filled columns** - Wastes time. Only check left (columns 0 to `col-1`).
11. **Printing during recursion** - Solution prints in `main()` after solver returns, not during recursion.
12. **Assuming all N have solutions** - $N=2,3$ have no solutions. $N=4$ have solutions.

Quick Revision Summary

N-Queens: Place N queens on $N \times N$ board, no two attack (rows/columns/diagonals).

Backtracking: Recursive exploration with pruning. Try each row in column, check safety, place queen, recurse next column, backtrack if dead-end. $O(N!)$ but efficient pruning.

Branch & Bound: Optimize with constraint lookup arrays (rowlookup, slashlookup, backslashlookup) for $O(1)$ safety checks instead of $O(N)$. Same recursion, much faster. Both undo placements when backtracking. Column-by-column reduces search space dramatically.

Experiment 5: Simple Chatbot (Question Answering System)

Title

Developing a Simple Chatbot: Rule-Based Question Answering System

Aim

To understand and implement a simple rule-based chatbot that responds to predefined questions using string matching and conditional logic (without using AI or machine learning techniques).

Theory / Concept

What is a Chatbot?

A chatbot is a software application designed to simulate conversation with users through text or voice. It can: - Answer questions - Perform tasks - Provide information - Engage in dialogue

Chatbots are used in customer service, information retrieval, entertainment, and education.

Key Concepts:

1. Types of Chatbots: **A) Rule-Based Chatbot (Used in this experiment) - Definition:** Responds based on predefined rules and patterns - **Mechanism:** Pattern matching using if-else statements - **Input Matching:** Compares user input with predefined question patterns - **Responses:** Fixed predefined responses for each pattern - **Advantages:** - Simple to implement - Fully controllable and predictable - Works offline - Cheap to develop - No training data needed - Deterministic output - **Limitations:** - Cannot handle variations (typos, phrasing) - Limited to predefined questions - Cannot learn from interactions - Breaks with unexpected input - Requires manual rule creation - Not scalable to large question sets

B) AI-Based Chatbot (Not used here) - Definition: Uses Machine Learning/NLP to understand and respond - **Mechanism:** Neural networks, language models, NLP algorithms - **Input Matching:** Semantic understanding (understands meaning, not just words) - **Responses:** Generated dynamically or retrieved from trained models - **Advantages:** - Handles variations naturally - Can learn from interactions - More natural conversations - Scalable - **Limitations:** - Requires large training data - Expensive to develop - Needs computational resources - Black-box behavior (hard to predict) - May hallucinate or give wrong answers

2. How Rule-Based Chatbot Works: Step-by-Step Process:

1. Start chatbot
2. Display available questions to guide user
3. Loop until exit:
 - a. Read user input
 - b. Convert to lowercase (normalize)
 - c. Check against predefined rules (if-elif-else)
 - d. If match found: print predefined response
 - e. If no match: print "I don't understand"
 - f. If "quit" input: exit loop
4. End chatbot

Example Flow:

```
User Input: "Hi"
↓
Convert to lowercase: "hi"
↓
Check rules:
- "hi" == "hi"? → YES
↓
Print Response: "Hello"
```

3. Key Design Principles:

- A) Case Normalization:** - Convert input to lowercase using `.lower()` - Why important: - “Hi” vs “HI” vs “hi” should all work - Users type differently; normalize for consistency - Simplifies pattern matching

- B) Predefined Question List:** - Display available questions upfront - Guides user on what to ask - Reduces user confusion - Sets chatbot boundaries - Improves user experience

- C) Pattern Matching:** - Exact string matching (simple but rigid) - Alternative: substring matching or fuzzy matching - This code uses: `question == "pattern"` or `question in [list]`

- D) Conversation Flow:** - Loop continues until user quits - Each interaction independent - State can be maintained (like name in this code)

4. Real-World Applications:

Application	Example
Customer Service	FAQ bots, support ticketing
E-Commerce	Product recommendations, order tracking
Banking	Account queries, balance checks
Healthcare	Symptom checkers, appointment booking
Education	Tutoring, Q&A systems

Application	Example
Entertainment	Game characters, trivia bots

5. Limitations of Rule-Based Approach:

1. **Scalability:** Hard to manage 1000s of rules
2. **Rigidity:** Exact matching only (no fuzzy matching)
3. **Maintenance:** Every change requires code modification
4. **User Experience:** Can frustrate users with “I don’t understand”
5. **Context:** Cannot maintain long-term conversation context
6. **Personalization:** Difficult to personalize responses

Algorithm

Rule-Based Chatbot Algorithm

Algorithm SimpleChatbot():

Input: User text input from keyboard

Output: Appropriate chatbot response

1. Display welcome message
2. Display list of available questions:
 - Print all predefined patterns/questions
 - Guide user on what to ask
3. Initialize:
 - Loop flag = True
4. While Loop flag is True:
 - a. Prompt: "Enter your question: "
 - b. Read user input
 - c. Convert input to lowercase:


```
normalized_question = input.lower()
```
 - d. Pattern Matching (if-elif-else):


```
If normalized_question == "hi":
    Print "Hello"

Else If normalized_question in ["how are you?", "how do you do?"]:
```

```
Print "I am fine"
```

```

Else If normalized_question in ["are you working?", "are you doing any job?"]:
    Print "Yes, I am working"

Else If normalized_question == "what is your name?":
    Print "My name is Emilia"
    Prompt: "Enter your name: "
    Read: user_name
    Print "Nice meeting you", user_name

Else If normalized_question == "what did you do yesterday?":
    Print "I saw Bahubali 5 times"

Else If normalized_question == "quit":
    Print "Goodbye!"
    Loop flag = False (exit loop)

Else:
    Print "I don't understand. Please choose from given questions."

5. End Program

End Algorithm

```

Code Explanation

Part 1: Welcome and Instructions

```

print("Simple Question and Answering Program")
print("=====")

print("\nYou can ask the following questions:")
print("1. Hi")
print("2. How are you?")
print("3. Are you working?")
print("4. What is your name?")
print("5. What did you do yesterday?")
print("6. Quit\n")

```

Purpose: - Display welcome message - List all available questions - Set user expectations - Guide user on what to ask - Prevent confusion with "I don't understand"

Part 2: Main Loop

```
while True:
    question = input("Enter your question: ").lower()
```

Logic: - while True: creates infinite loop - Loop breaks only when “quit” input received - .lower() converts input to lowercase immediately - **Why lowercase?** Normalize input: “Hi”, “HI”, “hi” all become “hi” - Important for pattern matching to work regardless of capitalization

Part 3: Pattern Matching - Simple Questions

```
if question == "hi":
    print("Hello")

elif question in ["how are you?", "how do you do?"]:
    print("I am fine")

elif question in ["are you working?", "are you doing any job?"]:
    print("Yes, I am working")
```

Logic: - if question == "hi": exact match single pattern - elif question in [list]: match ANY pattern in list - List approach allows multiple valid inputs - **In vs ==:** - == for single exact pattern - in for multiple patterns

Example Flow:

User types: "HI"
→ .lower() converts to "hi"
→ if question == "hi": → TRUE
→ print("Hello")

Part 4: Interactive Question - Name

```
elif question == "what is your name?":
    print("My name is Emilia")
    name = input("Enter your name: ")
    print("Nice meeting you", name)
```

Logic: - Print chatbot name - Ask user for name (store in variable) - Personalized response using user’s name - **State Management:** Variable name temporarily stores user input - This creates interactive conversation feel

Example:

Chatbot: "My name is Emilia"
User: "Alice"
Chatbot: "Nice meeting you Alice"

Part 5: Fixed Response Questions

```
elif question == "what did you do yesterday?":  
    print("I saw Bahubali 5 times")
```

Logic: - Simple pattern match - Print predefined response - No follow-up interaction needed

Part 6: Exit Condition

```
elif question == "quit":  
    print("Goodbye!")  
    break
```

Logic: - break statement exits while True loop - Program terminates gracefully - Goodbye message printed

Part 7: Default Case - Unmatched Input

```
else:  
    print("I don't understand. Please choose from the given questions.")
```

Logic: - Catches any input not matching predefined patterns - Reminds user of available options - Prevents silent failures - User knows chatbot didn't understand

Complete Execution Example:

Output:
Simple Question and Answering Program
=====

You can ask the following questions:

1. Hi
2. How are you?
3. Are you working?
4. What is your name?
5. What did you do yesterday?
6. Quit

```
Enter your question: hi  
Hello  
Enter your question: how are you?  
I am fine  
Enter your question: what is your name?  
My name is Emilia  
Enter your name: Alice  
Nice meeting you Alice
```

Enter your question: hello world
I don't understand. Please choose from the given questions.
Enter your question: quit
Goodbye!

Key Points / Important Notes

Rule-Based Chatbot Points:

- **Simple implementation:** Just if-elif-else statements
- **No AI needed:** Uses pattern matching only
- **Predictable behavior:** Same input always gives same output
- **Offline capability:** Works without internet
- **Easy debugging:** Logic is straightforward
- **Scalable limitation:** Hard with 1000s of rules
- **Limited flexibility:** Cannot handle variations or typos

Implementation Points:

- **Use `.lower()` for normalization:** Essential for case-insensitive matching
- **Display available questions:** Guides user and reduces errors
- **Use `in` for multiple patterns:** Allows question variations
- **Use `break` to exit loop:** Graceful termination
- **Default case (`else`):** Handles unexpected input
- **Interactive elements:** Can ask follow-up questions (like name)

Comparison: Rule-Based vs AI-Based:

Aspect	Rule-Based	AI-Based
Implementation	If-else, string matching	ML models, NLP
Understanding	Pattern matching	Semantic understanding
Flexibility	Low (exact match)	High (fuzzy matching)
Learning	No (fixed rules)	Yes (learns from data)
Development Time	Hours/Days	Weeks/Months
Maintenance	Manual rule updates	Automatic (model retraining)
Cost	Low	High
Scalability	Low	High
Accuracy	Good (for known questions)	Better (for variations)

Chatbot Scope & Limitations:

- This chatbot can handle **exactly 5 questions** + quit
- Adding more questions requires code modification

- Cannot understand synonyms or paraphrasing
- Cannot maintain memory across sessions
- No context awareness
- No learning capability

Complexity:

- **Time:** $O(1)$ for simple matching, $O(n)$ if checking n patterns
 - **Space:** $O(n)$ for predefined patterns
 - **Much simpler than AI-based approaches**
-

Viva Questions & Answers

Basic Definition Questions:

Q1: What is a chatbot? A: A chatbot is a software application that simulates human conversation with users through text or voice input, providing responses to queries or performing tasks.

Q2: What is the difference between a rule-based and AI-based chatbot? A: Rule-based chatbots use predefined rules and pattern matching (if-else). AI-based chatbots use machine learning/NLP to understand meaning and generate responses dynamically.

Q3: Why is this chatbot called “rule-based”? A: Because it operates on predefined rules: “if user input equals X, respond with Y.” All behavior is hard-coded; no learning occurs.

Q4: What does `.lower()` do? A: Converts the entire input string to lowercase. Example: “HI”, “Hi”, “hi” all become “hi”. Important for case-insensitive matching.

Q5: Why is `.lower()` important in this chatbot? A: Users type differently (capitals, lowercase). `.lower()` normalizes input so “Hi”, “HI”, “hi” all match the pattern “hi”. Without it, only exact case matches.

Conceptual Questions:

Q6: What is pattern matching? A: The process of checking if user input matches any predefined question pattern. If match found, print corresponding response.

Q7: Why display available questions to the user? A: Guides user on what to ask. Reduces “I don’t understand” messages. Improves user experience. Sets chatbot boundaries.

Q8: What are the advantages of rule-based chatbots? A: Simple, predictable, fully controllable, offline capability, no training needed, deterministic,

cheap to develop.

Q9: What are the limitations of rule-based chatbots? A: Cannot handle variations/typos, limited to predefined questions, not scalable, cannot learn, rigid, requires manual maintenance.

Q10: When would you choose rule-based over AI-based? A: For small question sets, offline use, predictability requirement, budget constraints, or when high accuracy on exact patterns is needed.

Code-Related Questions:

Q11: What does `question = input("Enter your question: ").lower()` do? A: Reads user input from keyboard, converts it to lowercase, stores in variable `question`.

Q12: What does `question in ["pattern1", "pattern2"]` check? A: Checks if `question` equals ANY element in the list. Returns True if found, False otherwise.

Q13: Difference between `question == "hi"` and `question in ["hi"]`? A: `==` checks single exact pattern. `in` checks if value exists in list (can have multiple patterns). Both work, but `in` is used for multiple synonyms.

Q14: What does `break` do? A: Exits the `while True` loop immediately. Used when user enters “quit” to terminate program.

Q15: What happens if user input doesn’t match any pattern? A: The `else` block executes, printing “I don’t understand. Please choose from the given questions.”

Comparison & Analysis Questions:

Q16: Why is this chatbot NOT an AI chatbot? A: It doesn’t use machine learning, neural networks, or NLP. It only uses string matching and if-else logic. No learning from data.

Q17: If you wanted to handle “Are you working?” differently from “Are you doing any job?”, how? A: Remove from list and use separate if statements: `elif question == "are you working?": ...` and `elif question == "are you doing any job?": ...`

Q18: Can this chatbot understand “Hi there” if you ask “hi”? A: No, because “hi there” “hi” (exact match). To handle this, use substring matching: `if "hi" in question:` instead.

Q19: What if user enters “How Are You?” (with capitals)? A: `.lower()` converts to “how are you?” which matches the pattern in the list. Works correctly.

Q20: How would you add a new question to this chatbot? A: Add new elif block: `elif question == "new question": print("response")`

Practical Questions:

Q21: Why use a list for similar questions like ["how are you?", "how do you do?"]? A: Allows multiple variations to trigger same response. Improves user experience - different phrasings are accepted.

Q22: How would you make this chatbot interactive? A: Add follow-up questions (like name). Example: Ask user's name and include in response: `print(f"Hello {name}")`

Q23: What's the maximum number of questions this chatbot can handle efficiently? A: For practical purposes, 100-1000 rules still work fine with if-elif-else. Beyond that, use dictionary mapping for efficiency.

Q24: How would you improve this chatbot without using AI? A: Add synonyms list, use substring matching, maintain conversation history, add more questions, improve error messages.

Q25: Is this chatbot suitable for production use? A: Only for very simple tasks with small question set. For production: use AI-based or hybrid approaches for better user experience.

Common Mistakes

1. **Forgetting .lower() on input** - Input won't match patterns with different capitalization. "Hi" won't match "hi".
2. **Using = instead of == for comparison** - = assigns value; == compares. Syntax error if not careful.
3. **Forgetting break in quit condition** - Loop continues even after "quit" input. Program won't exit.
4. **Not handling input variations** - Only exact matches work. "Hi there" won't match "hi". Use list for synonyms.
5. **Using or instead of in for multiple patterns** - `question == "a" or question == "b"` works but `question in ["a", "b"]` is cleaner.
6. **Forgetting indentation** - Code inside if-elif-else must be indented. Python is whitespace-sensitive.
7. **Using input() without storing result** - Must store in variable: `question = input(...)` not just `input(...)`.
8. **Not displaying available questions** - Users don't know what to ask. Better UX with upfront list.
9. **Hardcoding response with wrong variable** - Using undefined variables in print statements causes errors.
10. **Using mutable default arguments** - Not applicable here, but common Python mistake.

11. **Confusing `else` with `elif`** - Must use `elif` for next condition check, `else` for default case only.
 12. **Not testing all patterns** - Must verify each pattern works with input.
-

Quick Revision Summary

Simple Chatbot (Rule-Based): Uses pattern matching with if-elif-else to respond to predefined questions. Input normalized with `.lower()` for case-insensitive matching. Displays available questions to guide users. No AI/ML used - just string matching. Easy to implement but not scalable. Good for small Q&A systems. Breaks with typos or paraphrasing. Interactive elements possible (like name input). Terminates on “quit” input.

Final Exam Preparation Checklist

Before Exam, Revise:

- ☐ All algorithm pseudocodes
- ☐ Code explanations with line-by-line breakdown
- ☐ Complexity analysis (time and space)
- ☐ Comparison tables between algorithms
- ☐ All 25+ viva questions per experiment
- ☐ Common mistakes section

During Practical Exam:

- ☐ Write clear, well-commented code
- ☐ Verify output matches expected results
- ☐ Test edge cases
- ☐ Explain algorithm before coding
- ☐ Submit clean, readable solution

During Viva:

- ☐ Answer conceptually before code details
 - ☐ Reference algorithm while explaining
 - ☐ Provide examples and visualizations
 - ☐ Admit if unsure (better than wrong answer)
 - ☐ Ask for clarification if question unclear
-

Summary of All Experiments

Exp	Name	Type	Time	Space	Best For
1	Graph Traversal	Search	$O(V+E)$	$O(V)$	Traversal, paths
2	A* Algorithm	Informed Search	$O(b^d)$	$O(b^d)$	Optimal paths, puzzles
3	Prim's MST	Greedy	$O(V^2)$	$O(V^2)$	Network design
4	N-Queens	Backtracking	$O(N!)$	$O(N)$	Constraint satisfaction
5	Simple Chatbot	Pattern Matching	$O(n)$	$O(n)$	Q&A systems, rules

Good luck with your practical exams! Study well, code carefully, and practice the viva questions!

Document created for LP-II Practical Examination Preparation All algorithms explained with student-friendly approach Questions and answers curated for comprehensive exam coverage